Algorithmic Robotics
COMP/ELEC/MECH 450/550
Project 3: Barking up a Random Tree

The documentation for OMPL can be found at http://ompl.kavrakilab.org.

Plan collision-free motions for the seven-joint Panda arm from Project 2. Each configuration contains seven joint angles. A geometric path moves the arm through a 3D environment while respecting joint limits and avoiding obstacles. Velocity, acceleration, and torque constraints are outside this formulation.

The goal of this project is to implement a simple sampling-based motion planner in C++ using OMPL, then tune its parameters, along with those of other planners, to study planning time and path quality. All three exercises use the Panda arm.

Work in pairs and submit one solution per pair. A Piazza post will help you find a partner. Read the README for build and run instructions. Use your Project 2 environment:

```
micromamba activate ompl_env
cd project3_2026
make
```

Deadlines

The project consists of two parts. For the first part (Exercises 1 and 2), submit your Random Tree Planner implementation and a short write-up. The deadline is **October 15, 2026**. For the second part (Exercise 3), submit your full codebase and complete report. The deadline is **October 20, 2026**.

Random Tree Planner

The *Random Tree Planner*, or RTP, grows a tree in the robot's configuration space. Initialize the tree with the starting configuration, then repeat:

1. **Select** a configuration q_a uniformly at random from the existing tree.
2. **Sample** a configuration q_b uniformly from the configuration space. With a small probability, select the goal configuration instead. This probability is the *goal bias*.
3. **Check** the straight-line motion from q_a to q_b in configuration space. If the motion is valid, add q_b to the tree with q_a as its parent.
4. Stop when the goal is reached or the planning time expires. Extract a path by following parent pointers.

Many sampling-based planners use these same primitives: selection, sampling, and local planning. Implement RTP as described above, with uniform tree-vertex selection and a check of the full connecting motion. Use `SpaceInformation::checkMotion()` to check intermediate configurations along an edge.

Collision Checking

The supplied C++ code uses the Panda model and collision geometry from Project 2. Forward kinematics transforms the arm's 59 collision spheres into world coordinates for each seven-joint configuration. The `is_valid` routine checks these spheres against environment spheres and oriented boxes. See `provided/CollisionChecking3D.h` and the supplied arm-planning code. The collision model covers robot–environment collisions; self-collision is outside this assignment's model.

Scenes are stored in YAML files. Positions, sphere radii, and box half-extents are in meters. Obstacle quaternions use $[x, y, z, w]$ order. Keep the supplied motion-checking resolution fixed when comparing planners.

Project exercises

1. Implement RTP in C++ using `src/RTP.h` and `src/RTP.cpp`. Derive from `ompl::base::Planner` and correctly implement `solve()`, `clear()`, and `getPlannerData()`. Return an exact solution when the goal is reached. If time expires, return an approximate path ending at the tree vertex closest to the goal.
   - Use OMPL's state-space, validity, and goal interfaces so that your planner is independent of the robot's geometry.
   - Use the supplied goal-bias setter to expose this parameter for experiments.
   - Manage the planner's allocated states and tree nodes. Both `clear()` and the destructor must release the planner's stored data.
2. Compute valid motion plans for the **seven-joint Panda arm** from Project 2 using your RTP planner. A configuration is $q = (q_1, \ldots, q_7)$, with joint angles in radians. The configuration space is the product of the seven bounded joint intervals:

$$\mathcal{C} = \prod_{i=1}^{7} [q_i^{\min}, q_i^{\max}] \subset \mathbb{R}^7.$$

   The supplied setup uses `ompl::base::RealVectorStateSpace(7)` with bounds from the Panda URDF. Joint values are ordered from `panda_joint1` through `panda_joint7`. The base and gripper are fixed, and local motions interpolate linearly between joint configurations within these bounds.
   - Use `plan_arm.py` to run your C++ planner through the provided Python binding. Forward kinematics, collision checking, and the binding are supplied.
   - Create **two custom 3D scenes** containing sphere and oriented-box obstacles. Use the files in `resources/scenes/` as examples. Choose valid start and goal arm configurations whose direct joint-space connection is blocked by an obstacle.
   - Visualize the arm, obstacles, and solution paths in **Viser**. Use `plan_arm.py --visualize` for playback after planning, or `visualize_path.py` to replay a saved JSON path. Include images of both scenes, their start and goal configurations, and example RTP paths in your report. Submit your scene files and saved paths.

3. Experiment with parameter settings for your RTP planner and OMPL's **RRT**, **RRT-Connect**, and **PRM**, all using the Panda arm. Add your configurations to `configs/planners.json` and run `benchmark.py`. The supplied C++ code uses `ompl::tools::Benchmark` to execute repeated trials, measure planning time, and record native OMPL benchmark logs. OMPL calls each planner's `clear()` method between trials.
   Use the five start–goal queries in `resources/scenes/clutter.yaml`, with five trials per query and parameter setting. Explore at least three settings per planner. Compare equal planning time limits first. Tune goal bias for RTP and RRT, range for RRT and RRT-Connect, and maximum neighbors for PRM. You may also vary the planning time limit for each planner.
   Evaluate the following four metrics:

   - **Planning time**: elapsed time in the planner's `solve()` call, measured by OMPL, including collision checking.
   - **Path length**: the sum of Euclidean joint-space edge lengths, in radians.
   - **Path smoothness**: `PathGeometric::smoothness()` evaluated on the raw path waypoints. Lower values indicate smoother paths under this geometric metric.
   - **Minimum clearance**: the minimum robot-sphere surface distance to an obstacle over sampled path edges, in meters. The supplied code records this as a custom benchmark property. OMPL's separate solution-clearance property measures mean clearance at path waypoints.

   Your analysis should:

   - Explain how parameter choices affect each planner's behavior.
   - Compare all four metrics and the exact-solution success rate across planners.
   - Identify trade-offs and support your conclusions with repeated trials, figures, and tables.
   - Identify the best parameter settings you found for each metric and explain the observed results.
   - Use `plan_arm.py --visualize` to inspect representative paths in Viser with your selected planner settings.

   Run `benchmark.py` to create `results/benchmark.db`. The script uses OMPL's own tool to convert its benchmark logs into this database. Upload the database to Planner Arena to compare results and download plots for your report. Consult the README for commands. We will also hold a leaderboard competition. Your leaderboard position will not affect your grade, and the winner will receive a prize. Details will be announced later.

Protips

- Study OMPL's RRT implementation and its state ownership: https://github.com/ompl/ompl/tree/main/src/ompl. For RTP, select tree vertices uniformly and attempt the full sampled motion. Nearest-neighbor selection, range-limited extension, rewiring, and storing intermediate edge states as tree vertices are outside the specified algorithm.
- Use `getPlannerData()` to expose the tree's vertices and edges, including start and goal labels. This is useful for inspecting and debugging your tree.
- Implement `clear()` carefully: OMPL's benchmark reuses planner instances across trials.
- The planning algorithm stays in C++. The supplied Python programs load scenes, launch planning and benchmarks, save results, and visualize paths.

Deliverables

Run `make clean`, zip the project directory as `Project3_<your NetID>_<partner NetID>.zip`, and submit it to Canvas. Your code must compile and run in a modern Linux environment with the course's OMPL environment. Include a README with build and run commands, your custom scenes, saved example paths, planner configurations, and raw experiment results. Submit a PDF report of 1–5 pages containing:

- **(2.5 points)** A succinct statement of the problem you solved.
- **(2.5 points)** A description of the Panda arm's geometry and its bounded seven-dimensional joint space.
- **(5 points)** Images of your two custom 3D scenes and a description of their start–goal arm configurations.
- **(5 points)** Viser images of paths generated by RTP in both scenes.
- **(5 points)** Summarize your experience implementing and testing RTP in Exercise 2. Describe its performance and the appearance of its solution paths.
- **(40 points)** Compare your RTP with RRT, RRT-Connect, and PRM from Exercise 3. Support your conclusions quantitatively with figures and tables. Compare planning time, path length, path smoothness, minimum clearance, and exact-solution success rate. Discuss parameter choices and trade-offs, and connect the observed differences to each planner's algorithm.
- **(5 points)** Rate each exercise's difficulty from 1–10, estimate the hours spent on each exercise, and describe the hardest part. State each partner's individual contribution.

Proofread your report and include the figures or tables needed to support your claims. Reports are evaluated on their analysis and clarity of presentation.

The implementation is worth **35 points**. Your RTP must compile, run, and generate valid arm motion plans with correct exact and approximate solution handling. Write succinct, well-organized C++ code with clear state ownership and comments explaining the implementation.